28 FEB 2026

AADK TASK 2

Art Space App: Design, States, & Test Plan

MindMatrix

1. Feature & UI Design

Main Feature of My Enhanced Art Space App

My enhanced Art Space app allows users to:

- Swipe or tap through artworks
- View artwork details
- Save artworks to favourites
- View saved favourites in a separate screen

The app will have three main screens:

Gallery Screen

- Large Image
- Artwork Title
- Artist Name
- Year
- Previous Button
- Next Button
- Heart Icon
- “View Details” Button

Details Screen

- Enlarged Artwork Image
- Title
- Artist
- Year
- Description
- Back Button

Favourites Screen

- List of saved artworks
- Clickable artwork cards
- Empty state message (“No favourites yet”)

Composable Structure (Labeled)

GalleryScreen()

Column

- Image()
- Text() → title
- Text() → artist
- Text() → year

Row

- Button("Previous")
- Button("Next")

IconButton(Heart)

Button("View Details")

DetailsScreen()

Column

- Image()
- Text() → title
- Text() → artist
- Text() → year
- Text() → description
- Button("Back")

FavouritesScreen()

LazyColumn

- ArtworkCard()

State Changes

- currentArtworkIndex changes when user taps Next/Previous.
- favouritesList updates when heart icon is tapped.
- showDetails toggles when “View Details” is tapped.
- Navigation state changes between Gallery, Details, and Favourites screens.

2. State Variables & Logic

State Variable	Type	Initial Value	Transition
currentArtworkIndex	Int	0	Next → +1, Previous → -1
favouritesList	MutableList	Empty List	Heart icon → add/remove
showDetails	Boolean	FALSE	View Details → true, Back → false
currentScreen	String	"Gallery"	Navigate to "Details" or

Logic Transitions

- Tap Next → currentArtworkIndex++
- Tap Previous → currentArtworkIndex--
- Tap Heart Icon:
  If artwork not in favourites → add
  If already in favourites → remove
- Tap View Details → showDetails = true
- Tap Back → showDetails = false

Edge Case Handling:

- If at last artwork → cycle to first
- If at first artwork and tap Previous → cycle to last
- Prevent duplicate favourites

3. Component & Testable Units

Unit 1: Next Button Advances Artwork

- UI Components: Button("Next"), Image, Text
- Trigger: Tap Next
- Expected Result: Image and text update to next artwork

Unit 2: Favourite Icon Toggles State

- UI Components: IconButton(Heart)
- Trigger: Tap heart icon
- Expected Result: Artwork added to favourites list OR removed if already added

Unit 3: Details Screen Shows Correct Artwork

- UI Components: View Details button, DetailsScreen composables
- Trigger: Tap “View Details”
- Expected Result: Details screen displays correct artwork information

Unit 4: Favourites Screen Displays Saved Items

- UI Components: LazyColumn
- Trigger: Navigate to Favourites screen
- Expected Result: Saved artworks appear in list

4. Test Plan Table

Test ID	Unit Under Test	Test Type	Preconditions	Trigger	Expected Outcome
T1	Next Button	UI Test	App launched, index = 0	Tap Next	Image and title update to index 1
T2	Previous Cycling	UI Test	Index = 0	Tap Previous	Index changes to last artwork
T3	Add to Favourites	UI + Unit	Artwork not in favourites	Tap Heart	Artwork added to favourites list
T4	Prevent Duplicate	Unit Test	Artwork already in favourites	Tap Heart again	Artwork removed OR no duplicate
T5	Details Screen Data	UI Test	On Gallery screen	Tap View Details	Details screen shows correct
T6	Empty Favourites	UI Test	No favourites added	Open Favourites	Show “No favourites yet”
T7	Orientation Change	UI Test	On artwork index 2	Rotate device	Same artwork still displayed

Normal Cases Covered

- Button navigation works
- Favourites update correctly
- Details screen loads proper data

Edge Cases Covered

- Cycling at boundaries
- Duplicate favourites
- Empty state
- Orientation change

5. Reflection

Planning tests alongside design makes the app more reliable because I clearly define expected behavior before coding. It helps me think about edge cases early instead of fixing bugs later. I think writing business logic unit tests will be easier because they don't depend on UI rendering. UI tests might be harder due to recomposition and state updates. To reduce flakiness in UI tests, I will use stable test tags, avoid relying only on text matching, and ensure state is properly initialized before each test.